

robbit Development Manual

Kise Laboratory
Created: September 2025

robbit: Overview

- robbit is an accessible two-wheeled self-balancing robot that uses an FPGA.

- Development Workflow

Assemble robbit



Generate the Bitstream and Binary

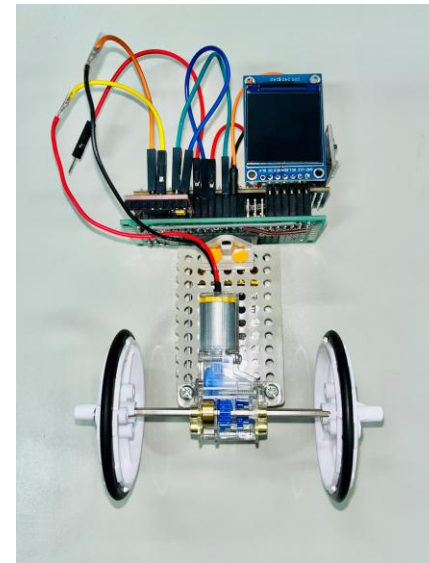


Operation Check



Parameter Tuning

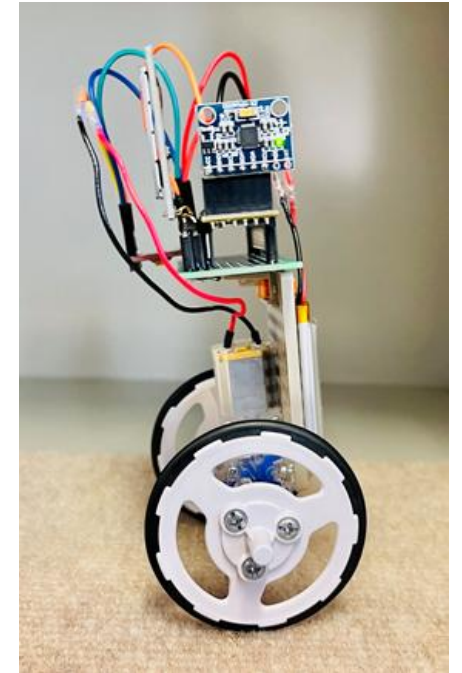
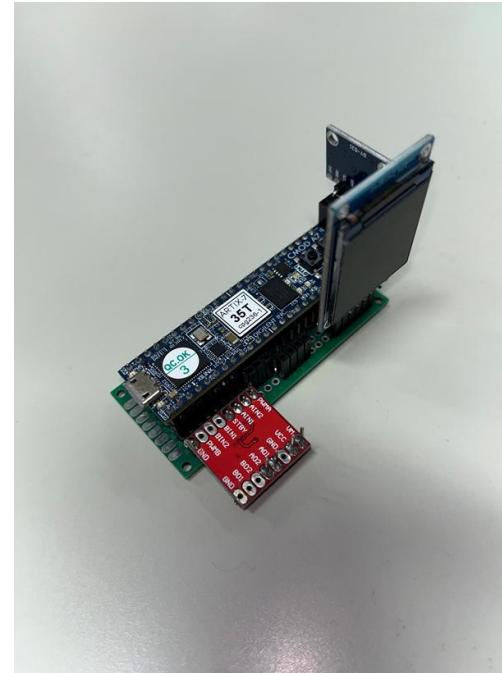
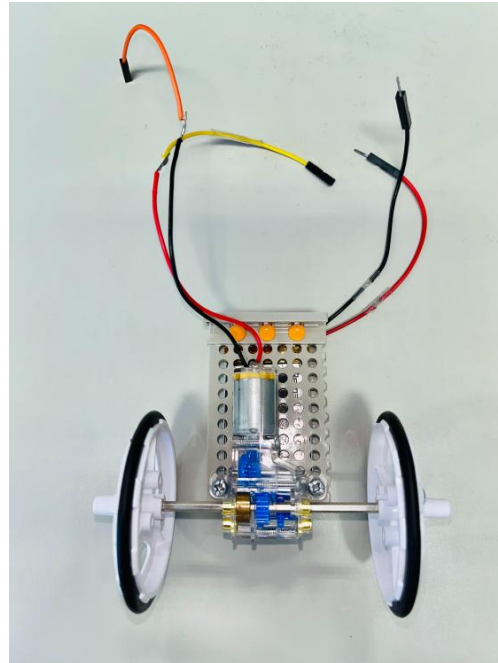
robbit



Assembling robbit: Purchase Components

- Assemble robbit using the following steps.

Purchase Components → Build the Chassis → Build the Control Unit → Assembly



Assembling robbit: Purchase Components

■ Purchase the components listed below.

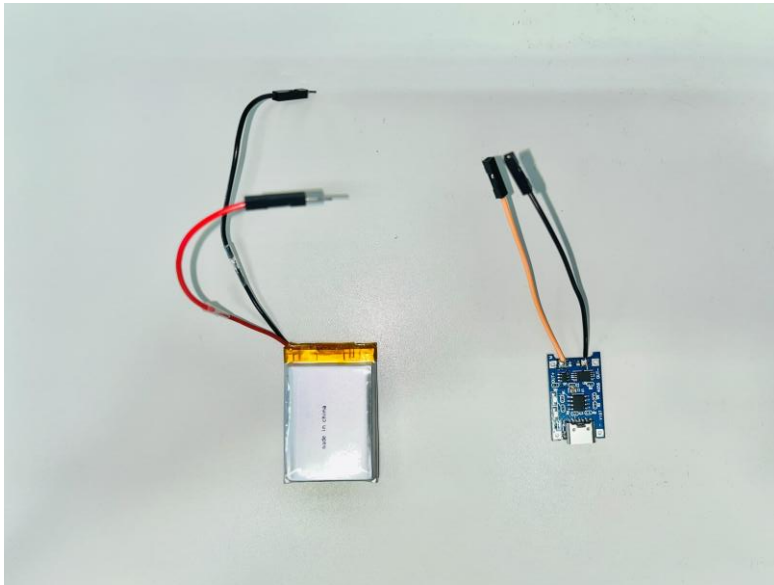
Component	Product	Quantity
FPGA Board	Cmod A7-35T	1
Sensor	MPU-6050	1
Motor	Mini Motor Standard Gearbox 70188	1
Tires	Slim Tire Set (55mm Dia.) 70193	1
Motor Driver	TB6612FNG	1
Battery	EEMB Lithium-Ion Battery 653042	1
Plate	Universal Plate Set 70157	1

- **Make sure that all the components shown below are available.**

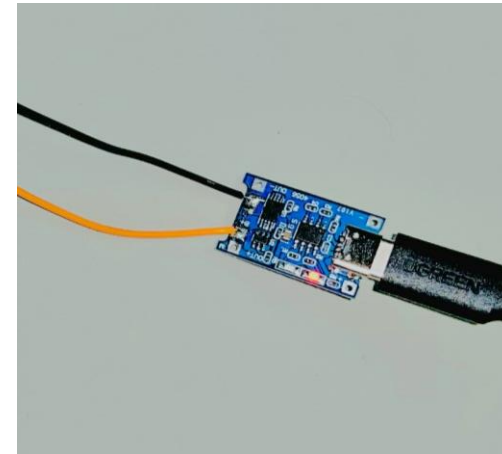


Assembling robbit: Build the Chassis

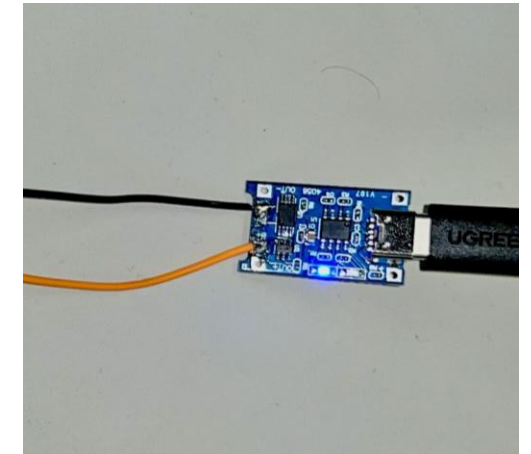
- Solder the battery and charging module as shown.
- After soldering, connect the battery to the charging module.
- The charging-module LED indicates the battery status.
(Red: charging; blue: fully charged)



Soldering Example



Red: Charging



Blue: Fully Charged

Charging-Module LED

Assembling robbit: Build the Chassis

- Cut the universal plate to size.
- Size the universal plate to match the control unit that will be built later.
- Leave an area for attaching the control unit, as shown below.



Universal Plate After Cutting



Control-Unit Attachment Area

Assembling robbit: Build the Chassis

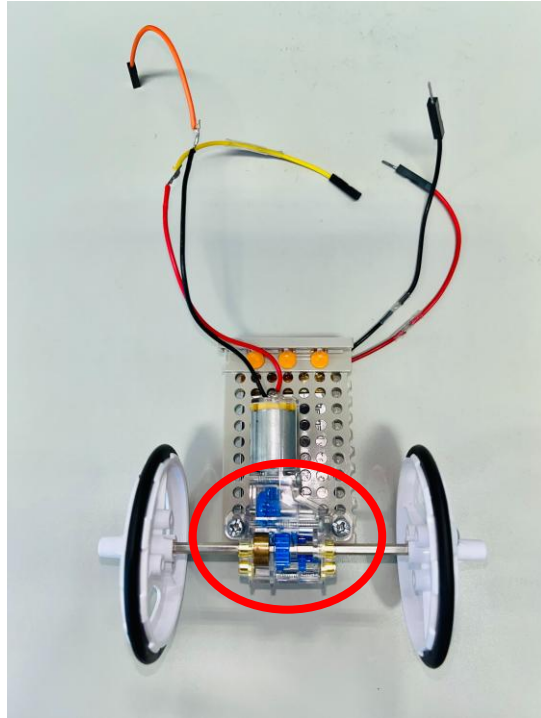
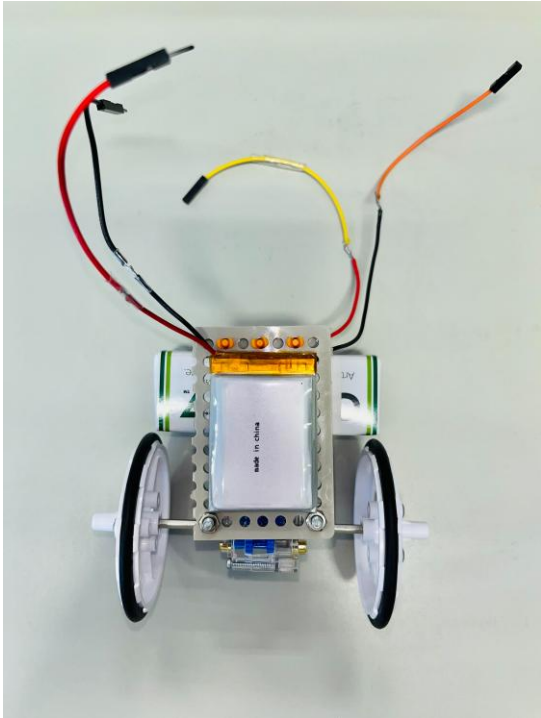
- Assemble the TAMIYA Mini Motor Standard Gearbox 70188 and Slim Tire Set (55 mm Dia.) 70193.
- The gearbox assembly instructions are shown on the next page. Required components and steps are highlighted in yellow.
- We recommend starting with the 55 mm tires. Smaller tires are also included, but they may make control more difficult.

Institute of
SCIENCE TOKYO

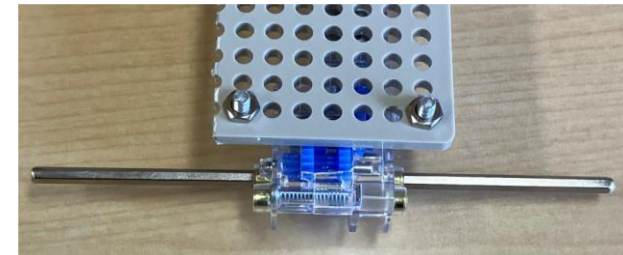
[illegible]

Assembling robbit: Build the Chassis

- Attach the tires after assembling the motor.
- Finally, attach the battery and motor to the universal plate to complete the chassis.



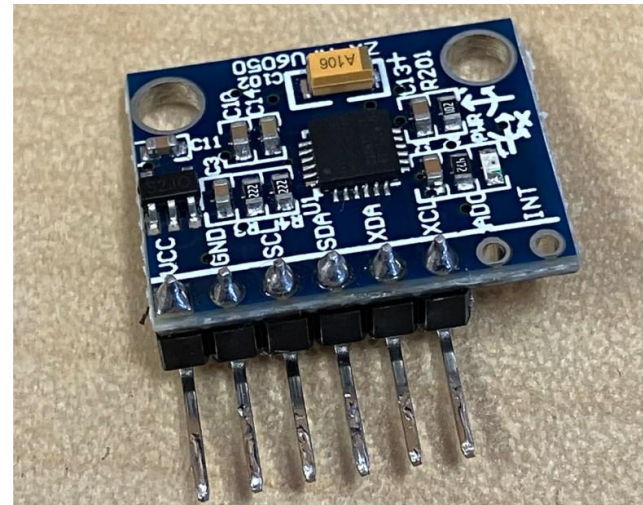
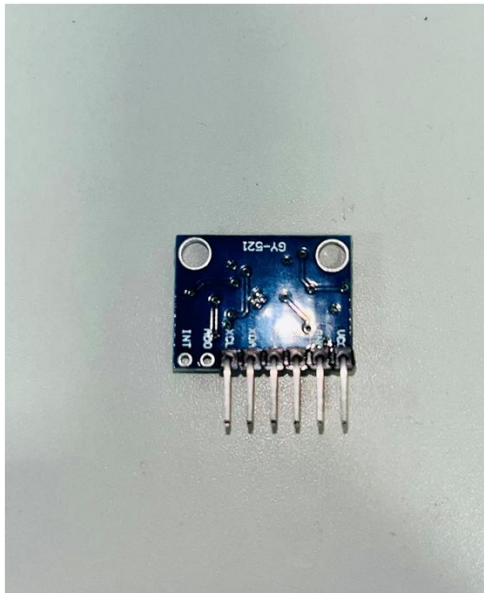
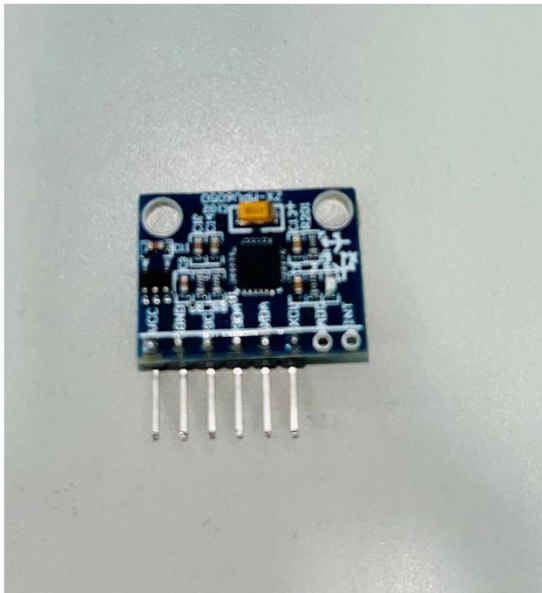
Fasten Securely with Screws



Completed Chassis

Assembling robbit: Build the Control Unit

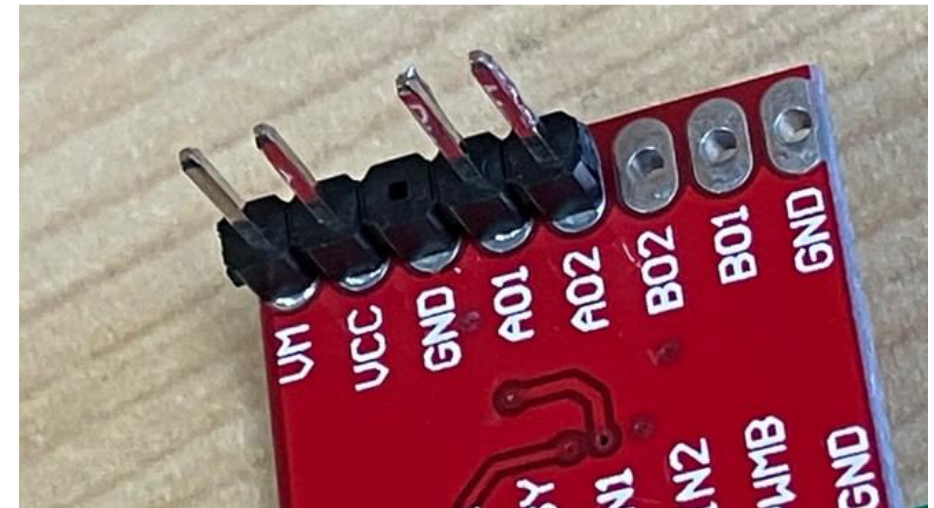
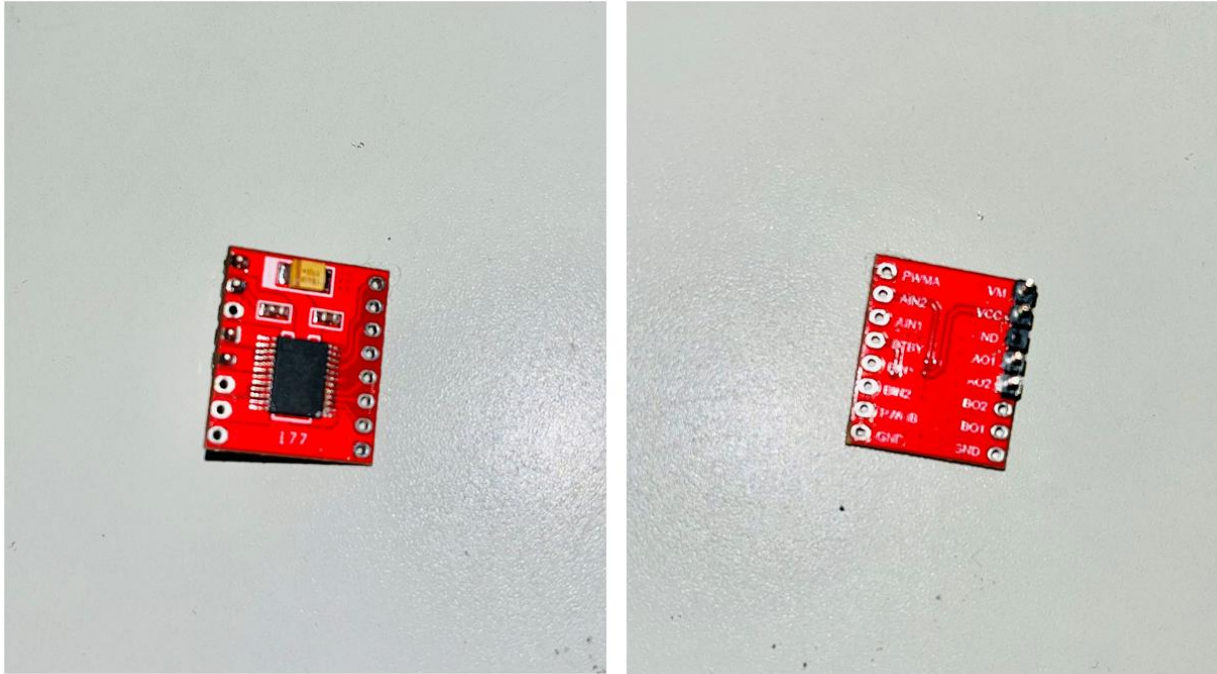
- Solder pin headers to the MPU-6050 sensor. Although the IMU operates with VCC, GND, SCL, and SDA connected, soldering headers to every connection point is recommended for better mechanical stability.



Soldering Example

Assembling robbit: Build the Control Unit

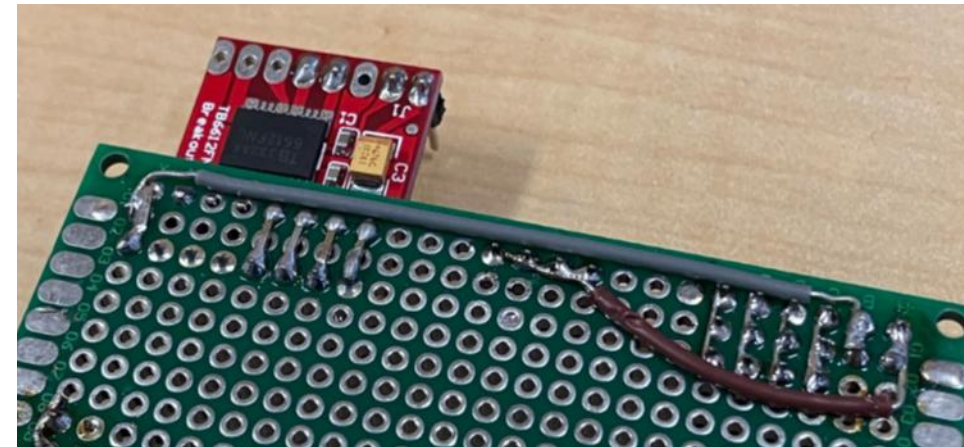
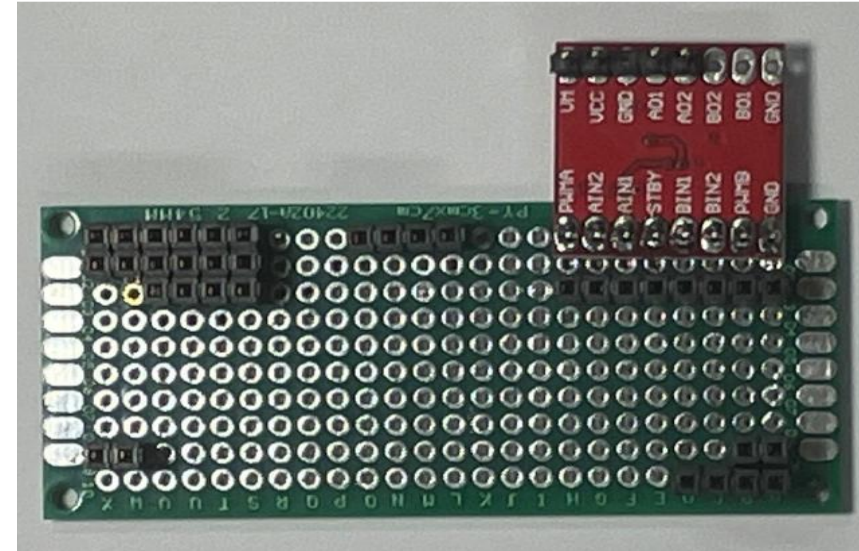
- Solder headers or wires to VM, VCC, GND, AO1, and AO2 on the TB6612FNG motor driver.
- Remove the unused GND pin.



Soldering Example

Assembling robbit: Build the Control Unit

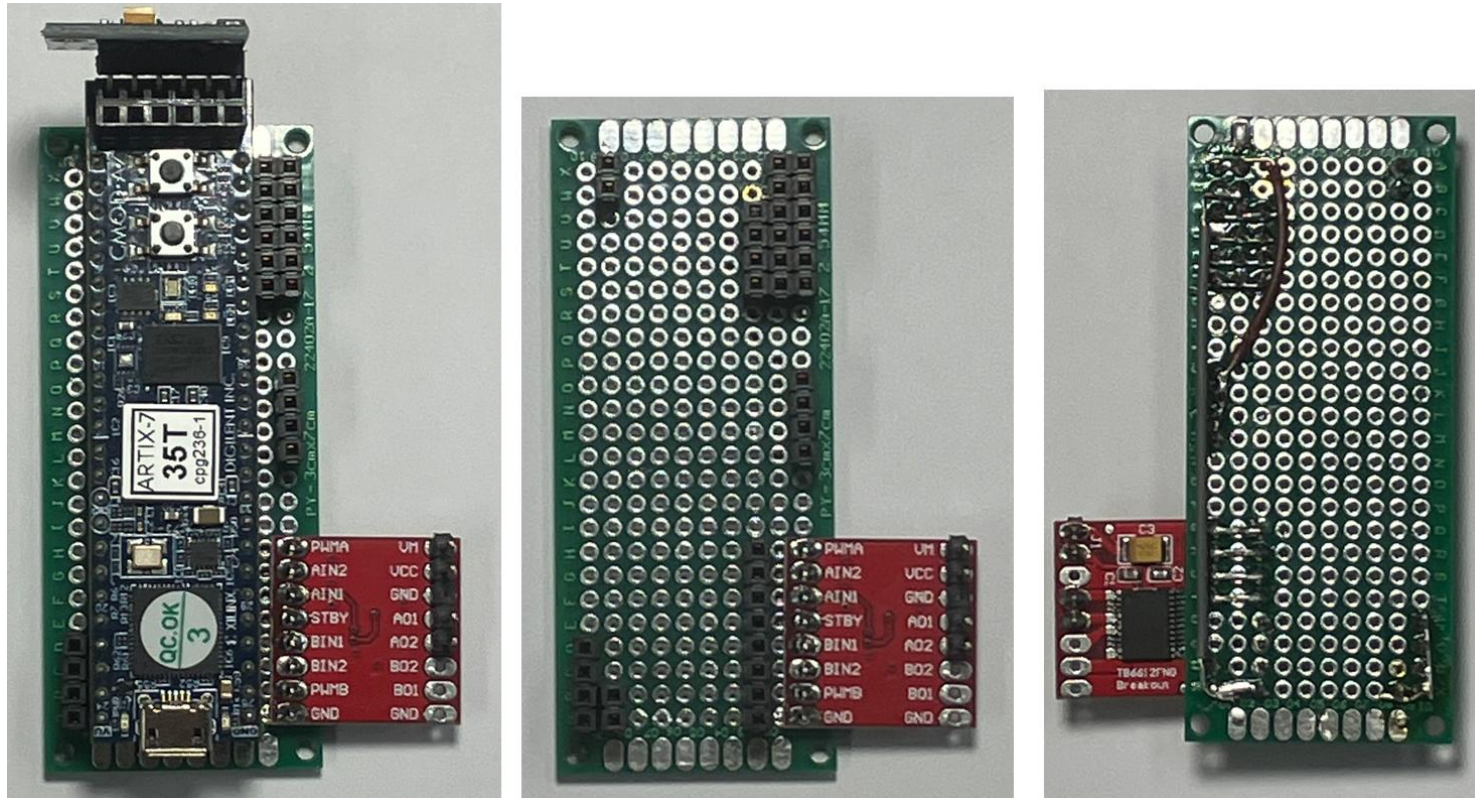
- Cut the pin connectors to the appropriate length and solder them to the universal PCB.
- Insert the black connectors shown in the photo into the universal PCB and solder them from the back.
- Solder the motor driver to the PCB as well.



Soldering Example

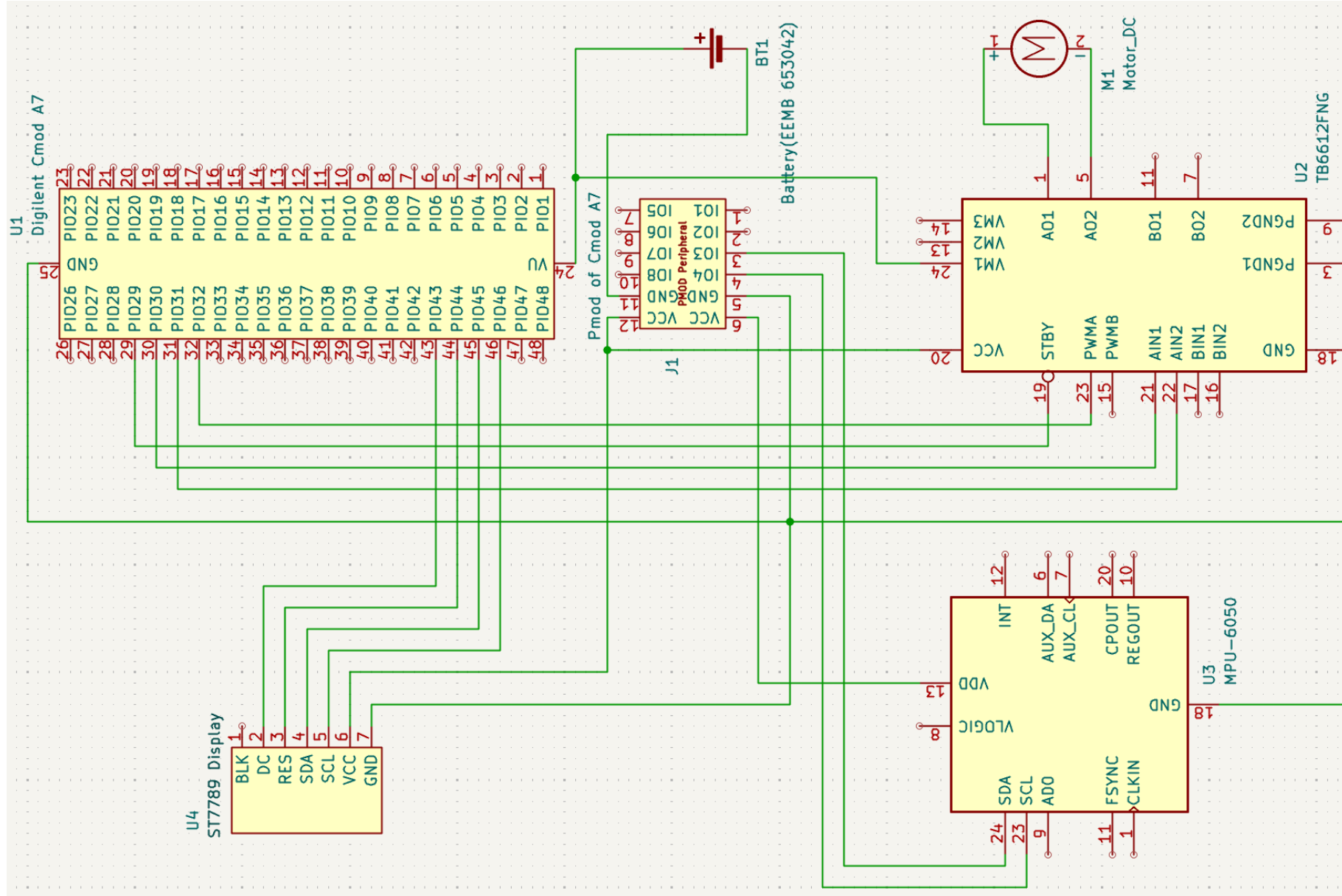
Assembling robbit: Build the Control Unit

- Prepare a breadboard or universal PCB.
- Wire and solder the prepared board using the image below and the circuit diagram on the next page.



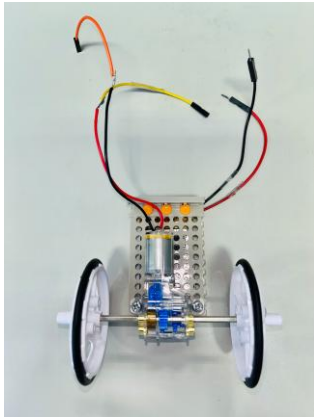
Control-Unit Soldering Example

Assembling robbit: Build the Control Unit

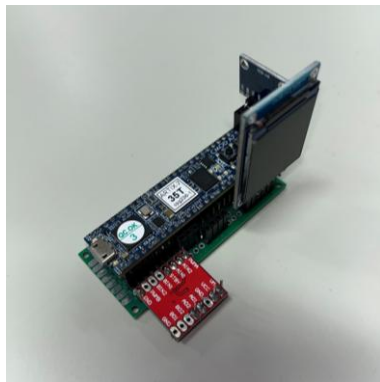


Assembling robbit: Build the Control Unit

- Attach the completed chassis and control unit with double-sided tape, then connect the wiring to complete robbit.



Chassis



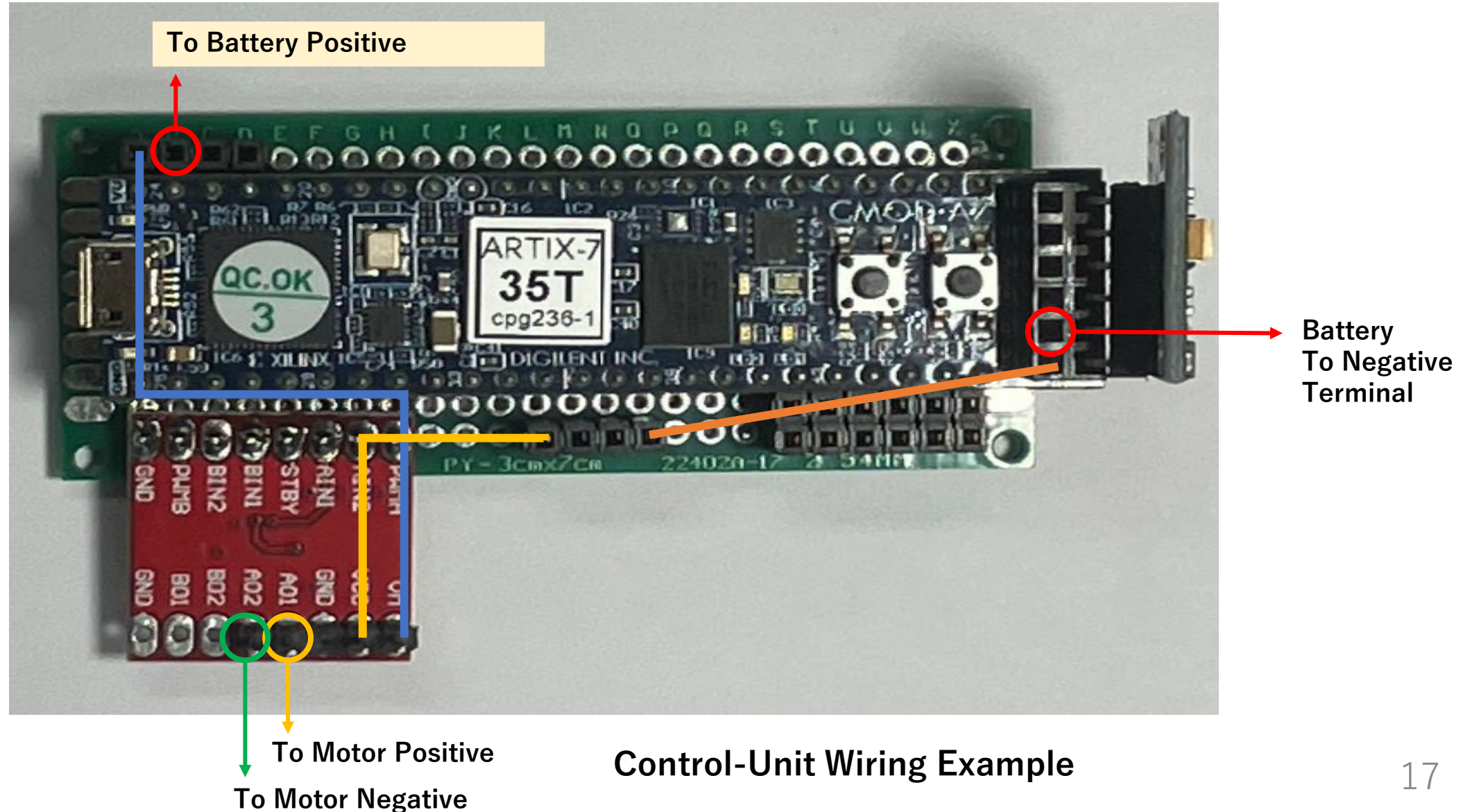
Control Unit

Assembly



robbit

Assembling robbit: Build the Control Unit



Bitstream and Binary Generation: Environment Setup

- Clone the robbit repository to the local environment using the command below.

```
git clone git@github.com:archlab-sciencetokyo/robbit\_project.git
```

- Because CFU Proving Ground is included as a submodule, move to the robbit_project directory and initialize and update it using the following command.

```
git submodule update --init --recursive
```

Bitstream and Binary Generation: Environment Setup

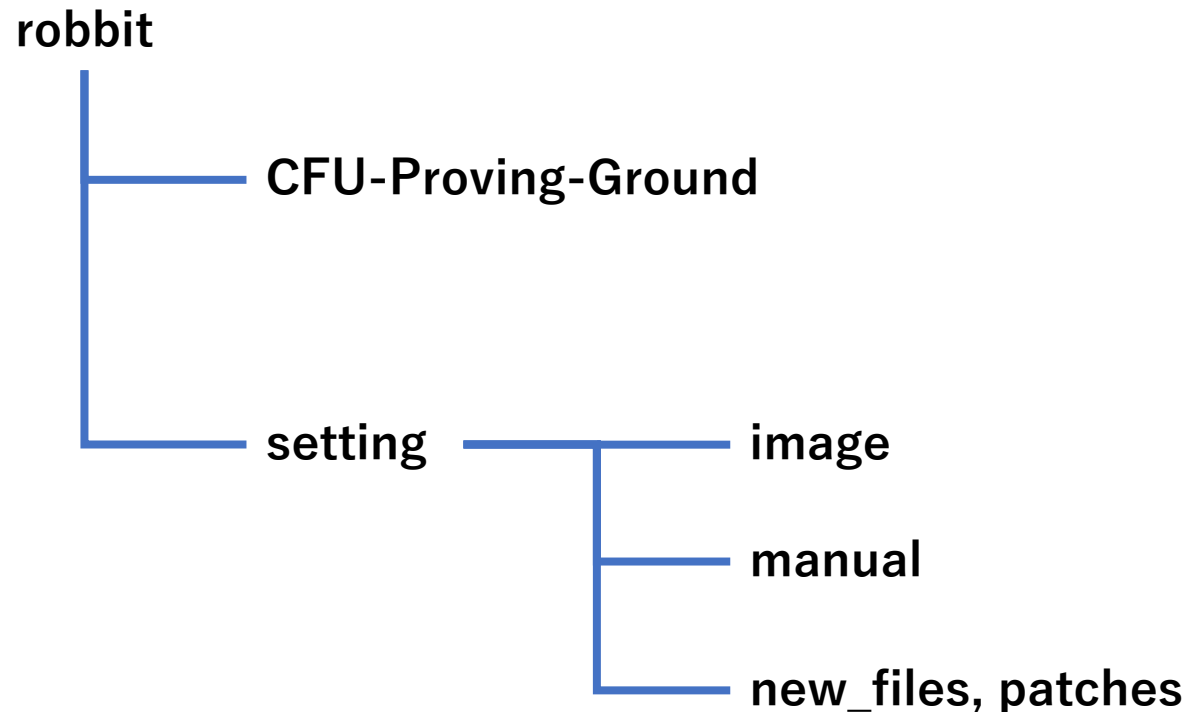
If submodule initialization (``git submodule update --init --recursive``) fails, try the commands below.

- This procedure is useful when the referenced CFU Proving Ground commit is not available locally.

```
cd robbit_project/robbit/CFU-Proving-Ground (move to the submodule directory)
git fetch
cd ../../ (return to the robbit_project directory)
git submodule update --init --recursive (run again)
```

Bitstream and Binary Generation: Environment Setup

- After CFU-Proving-Ground is initialized, the directory structure is as follows.
- Files under the setting directory normally do not need to be edited.
- Begin the environment setup in the robbit directory.



Bitstream and Binary Generation: Environment Setup

- Next, run the following command to prepare the robbit gateway and software for CFU Proving Ground.

make init

- The command copies main.cpp and the Verilog HDL files into the CFU-Proving-Ground directory.
- **Perform the remaining work in the CFU-Proving-Ground directory.**

Bitstream and Binary Generation: Environment Setup

The terminal output up to this point should look like the example below.

```
• kumagai@rserv5:~$ git clone git@github.com:archlab-sciencetokyo/robbit_project.git
Cloning into 'robbit_project'...
remote: Enumerating objects: 269, done.
remote: Counting objects: 100% (46/46), done.
remote: Compressing objects: 100% (38/38), done.
remote: Total 269 (delta 13), reused 28 (delta 8), pack-reused 223 (from 1)
Receiving objects: 100% (269/269), 63.30 MiB | 10.43 MiB/s, done.
Resolving deltas: 100% (125/125), done.
• kumagai@rserv5:~$ cd robbit_project/
• kumagai@rserv5:~/robbit_project$ git submodule update --init --recursive
Submodule 'robbit/setting/CFU-Proving-Ground' (https://github.com/archlab-sciencetokyo/CFU-Proving-Ground.git) registered for path 'robbit/CFU-Proving-Ground'
Cloning into '/home/kumagai/robbit_project/robbit/CFU-Proving-Ground'...
Submodule path 'robbit/CFU-Proving-Ground': checked out 'cdd36452d6aa41d2fe8568df16544d94ad51ca65'
• kumagai@rserv5:~/robbit_project$ cd robbit
• kumagai@rserv5:~/robbit_project/robbit$ make init
```



```
patching file app/st7789.c
Applying st7789.h.patch...
patching file app/st7789.h
Applying top.v.patch...
patching file top.v
Merge complete.
make[1]: Leaving directory '/home/kumagai/Work/bthesis/robbit/refecter/robbit_project/robbit'
• kumagai@rserv5:~/Work/bthesis/robbit/refecter/robbit_project/robbit$ cd CFU-Proving-Ground/
```

← Indicates that `make init` completed successfully

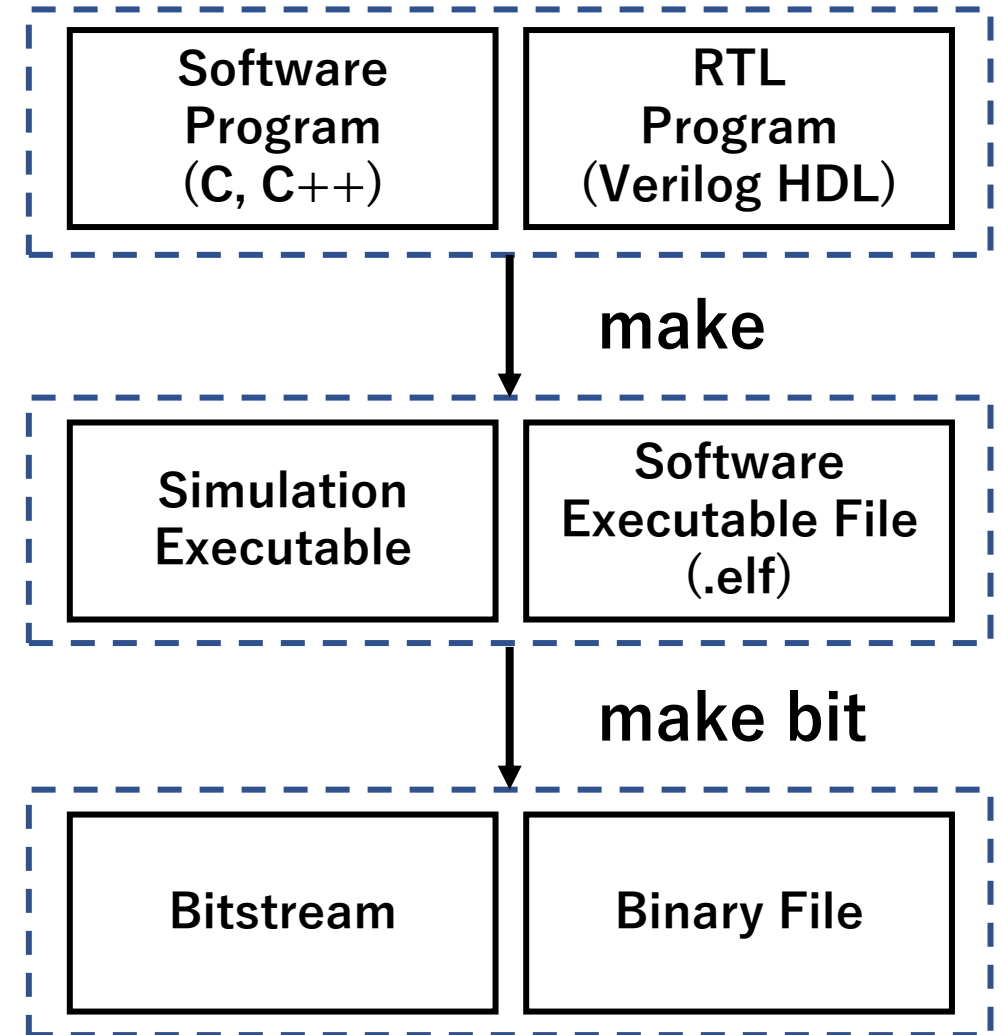
Continue working under CFU-Proving-Ground

Bitstream and Binary Generation: Environment Setup

- To set up the environment for CFU Proving Ground, follow Step 1 in its repository README.
- The setup requires tools such as a RISC-V compiler and AMD/Xilinx Vivado, and may take several hours.
- If local setup is difficult, an ACRI Room environment can reduce the setup effort.

Bitstream and Binary Generation: Build

- After setting up the environment, use the following workflow to build and generate the bitstream and binary files.
- Running through `make bit` generates both the bitstream and binary files.
 - **make**
 - **make bit**



Bitstream and Binary Generation: Build

- After bitstream generation finishes, check the timing-constraint result displayed in the terminal.
- A slack value of zero or greater means the timing constraints are satisfied.
- A negative slack value means the timing constraints are violated.

required time	4.516
arrival time	-4.484
slack	0.032

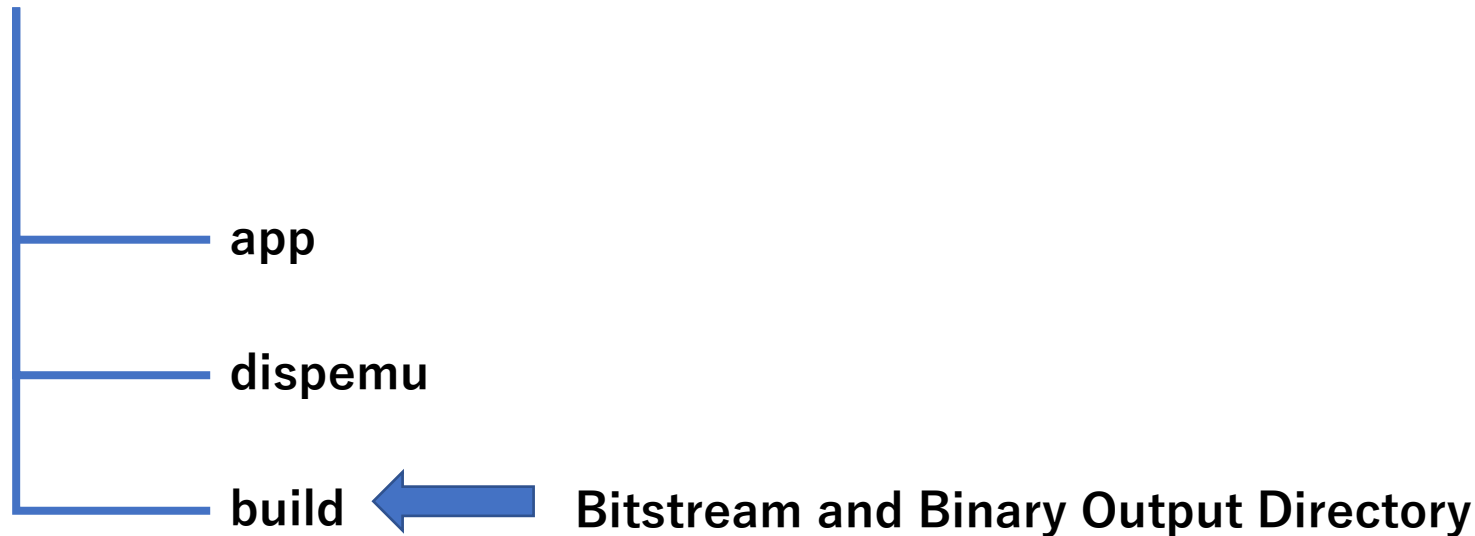
← The nonnegative value indicates no timing violation

- If the Timing Constraints Are Violated
 - Reduce the operating frequency (see page 34)
 - Optimize the critical path

Bitstream and Binary Generation: Build

- Running ``make bit`` creates a new build directory.
- The build directory contains the bitstream and binary files.

CFU-Proving-Ground

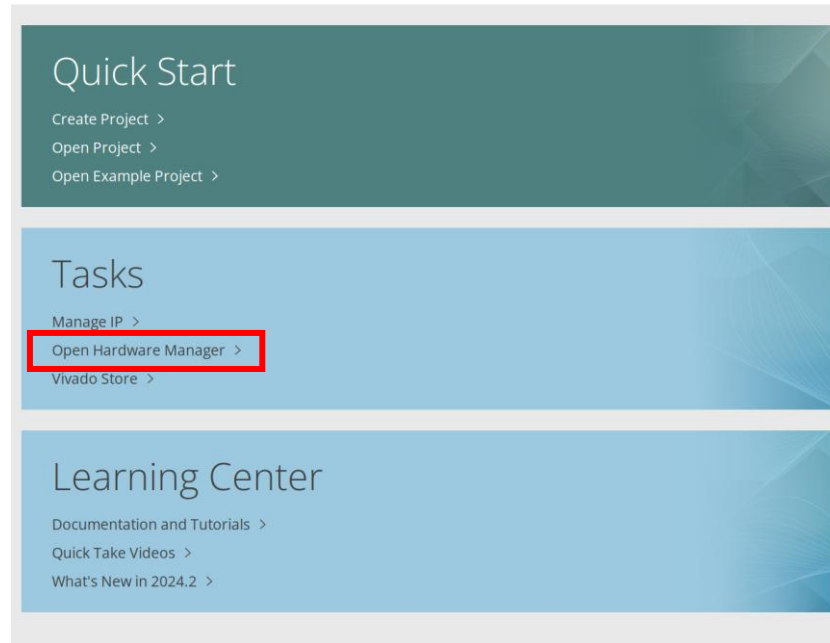


Bitstream and Binary Generation: Programming Setup

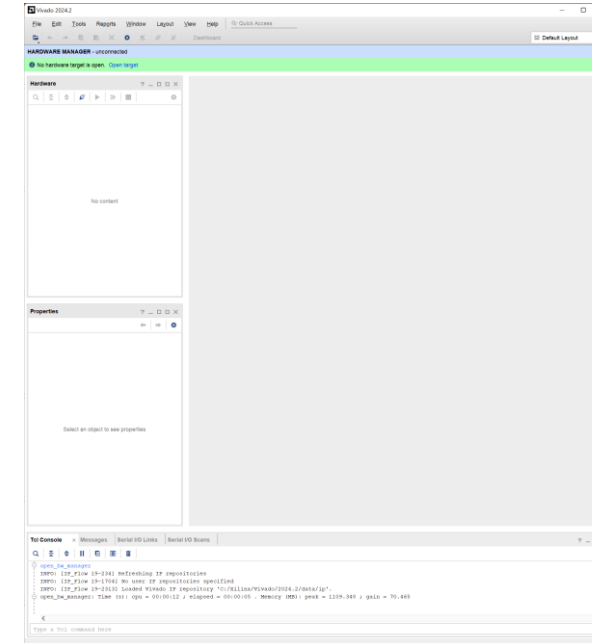
- Start Vivado and select Open Hardware Manager.



Vivado



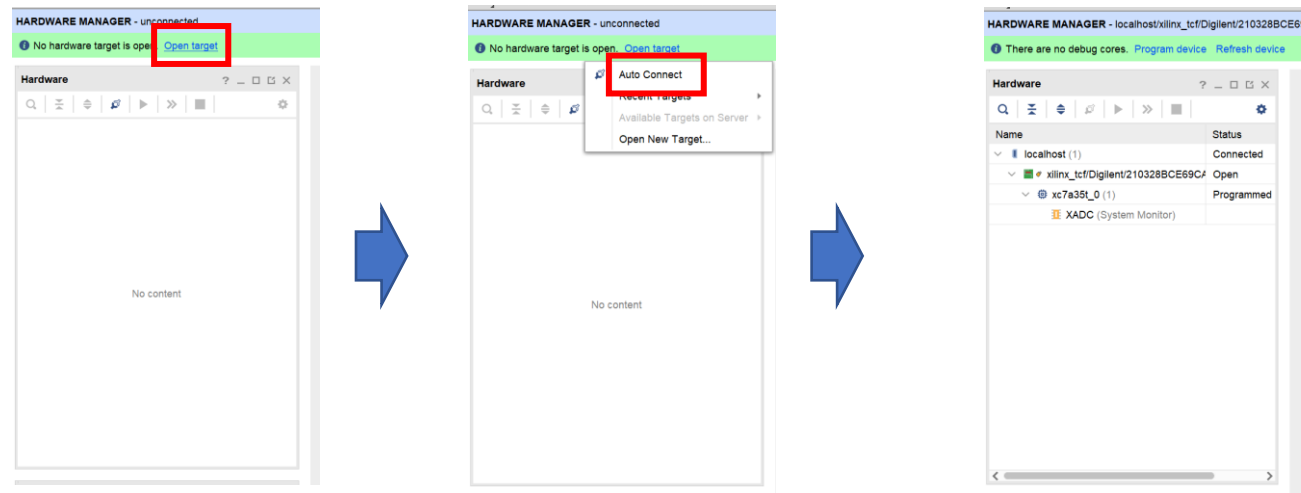
Vivado Start Screen



Hardware Manager

Bitstream and Binary Generation: Programming Setup

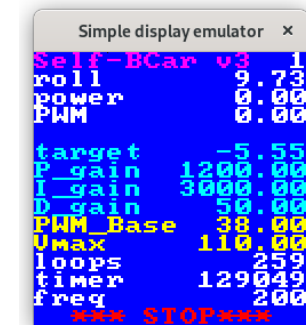
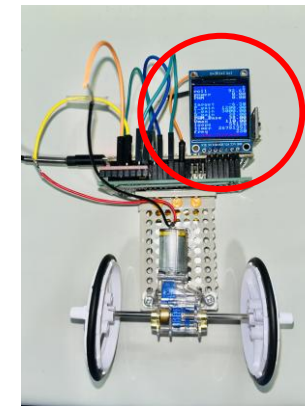
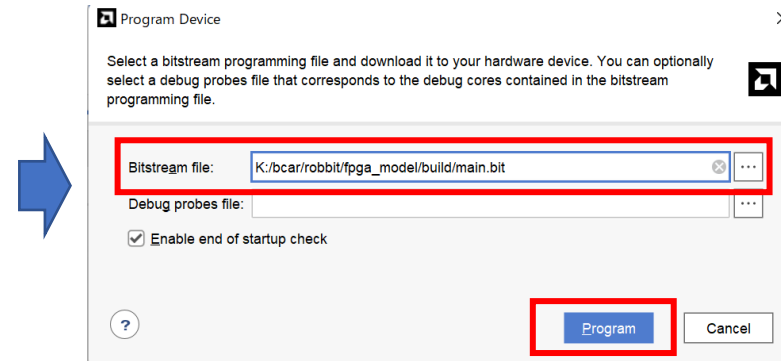
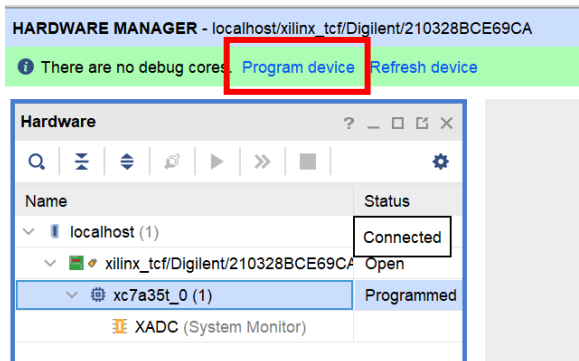
- Connect the PC to robbit's FPGA board with a cable.
- In the upper-right corner of Vivado, click `Open target` → `Auto connect`.
- When the board is detected, the FPGA appears as shown in the rightmost image below.



Example of a Successful Connection

Operation Check: Program the Bitstream

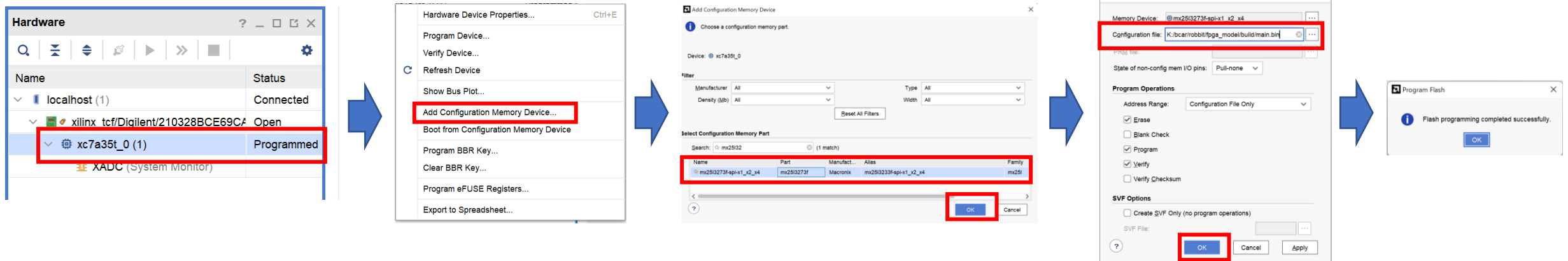
- Select Program device in the upper-right corner of Vivado.
- Select main.bit from the generated build directory, then click Program.
- After successful programming, robbit's display shows parameters, angle data, and other information.
- The bitstream configuration is lost when robbit is disconnected or powered off.



Example Display

Operation Check: Program the Binary File

- Program the binary file to operate robbit without a continuous PC connection.
- Right-click the FPGA and select `Add configuration memory device`.
- Search for `mx25l32`, select the resulting ROM device, and click OK.
- When prompted for a configuration file, select main.bin in the build directory and click OK to program it.



Operation Check

- After programming, disconnect robbit from the PC, turn it on, and verify its operation.
- Perform the operation check on a surface with some friction, such as carpet.
- Balancing in place is more difficult on a low-friction surface.

Operation Checklist

- When held upright, does the display show a roll value near 0° ?
- When robbit is tilted by hand, do the wheels rotate in the direction of the tilt?
- Do the wheels stop after the robot is tilted far from vertical? (The program stops them at 40° .)

Parameter Tuning: Controls

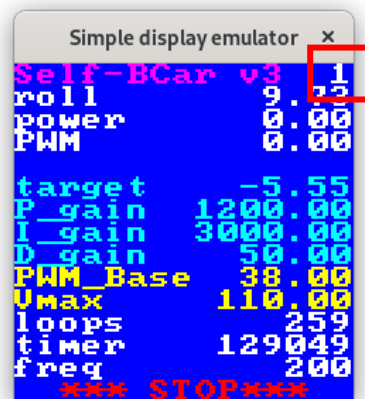
- Use the two buttons on the FPGA board to tune parameters.

BTN1: Decrease the parameter

BTN0: Increase the parameter

Press BTN1 and BTN0 together: Select another parameter

- Only one parameter can be changed at a time. The selected parameter is identified by the number shown on the display; this manual calls it the parameter number.



Selected Parameter Number

Tunable Parameters



Cmod A7-35T (FPGA Board)
Buttons

Parameter Tuning: Parameter Numbers

- Each tunable parameter is assigned a parameter number.

Parameter	Parameter Number	Description
Target	1	Target body angle
P_gain	2	Proportional gain
I_gain	3	Integral gain
D_gain	4	Derivative gain
PWM_Base	5	PWM signal offset
V_max	6	Maximum PWM signal value

Parameter Tuning: Other Parameters

- Additional parameters can be changed directly in the program.

- Operating Frequency

The CPU operating frequency can also be changed.

`CLK\_FREQ\_MHZ` in config.vh specifies the operating frequency.

Specify the value in MHz.

```
// cpu
`define CLK_FREQ_MHZ 150 // operating clock frequency in MHz
```

Example: 150 MHz

A value that is too high may violate timing constraints, so check the slack during bitstream generation.

Parameter Tuning: Other Parameters

- Loop Frequency

The loop frequency is the frequency of the while loop in main.cpp.

`LOOP\_HZ` in main.cpp specifies the loop frequency.

Increasing the value makes the loop execute more frequently.

Specify the value in Hz.

```
#define LOOP_HZ      1000 // Hz of main loop
```

Example: 1000 Hz

- Integral-Term Limit

Set an integral-term limit to implement anti-windup.

The value is set by `I\_MAX` in main.cpp.

```
#define I_MAX      0.4 // Anti-windup
```

Example: limit set to 0.4

Display Information

- The other values shown on the display are listed below.

Parameter	Description
roll	Current body angle
power	PID controller output
PWM	PWM signal value
loops	Number of loop iterations
timer	Clock count since startup
freq	Software loop frequency

Suggested Challenges

**Choose a performance goal for parameter tuning.
Examples are shown below.**

■ Difficulty: Easy

Balance for at least 10 minutes on a high-friction surface such as carpet.

■ Difficulty: Hard

**Balance for as long as possible on a low-friction surface.
Add disturbances such as wind or an incline.**